Database Technology, Management and Security

Project Type 4 — Experimental / Benchmarking Project

Do Common PostgreSQL Index Tuning Practices Actually Help?

Submitted By

Group XX

Name	Student ID
Imtiaj Sajin	XX-XXXXX-X

Submitted To

Dr. Ashraf Uddin

Assistant Professor

Department of Computer Science

American International University-Bangladesh

Submission Date: 22 August 2026

Declaration of Originality

I declare that this report is my own original work. All sources of information have been acknowledged and cited using the IEEE referencing style. No part of this report has been copied from any other student's work or from any other source except where explicitly cited. I understand that any act of plagiarism will be dealt with according to the university's academic integrity policy.

All measurements reported here were produced by the software published with this report [1]. Every figure and table is generated directly from the raw measurement files; no numerical value in this report was transcribed by hand.

Imtiaj Sajin

Abstract

Choosing the wrong index can make a query orders of magnitude slower, and a substantial body of practitioner guidance exists to help database administrators avoid that outcome. This report asks whether that guidance survives measurement. We built a controlled benchmark that varies value skew, predicate correlation and physical clustering independently, and measured 11,136 query executions against PostgreSQL 17.1 across two table scales, eleven dataset configurations, ten index configurations, seven table scales and seven cost-model settings. Ground-truth cardinality for every predicate is computed from the generated data rather than taken from the database, so estimation error is measured directly. The first result is reassuring: with a conventional index set, PostgreSQL selects a near-optimal access path, erring materially in only 2.3% of queries at a median penalty of 1.3%. The remaining results are not. Three widely recommended practices are evaluated and all three are found to backfire under identifiable conditions. Extended statistics improve cardinality estimates by up to 108 times while making the affected queries up to 2.7 times slower. Lowering `random_page_cost` below its default on solid-state storage makes the workload 2.3 times slower rather than faster. Adding a BRIN index alongside a B-tree on the same column raises misselection from 2.3% to 73.3%, with individual queries degrading by up to 194 times, while the table is resident in the buffer pool. Measuring five intermediate scales shows that this last effect does not simply vanish as the table grows: its frequency collapses once the heap exceeds `shared_buffers`, but its worst case doubles at that same point and subsides only once queries begin paying physical I/O. Each behaviour is traced to a specific mechanism visible in the query plan.

Keywords: query optimisation; access path selection; index selection; cardinality estimation; PostgreSQL; performance benchmarking

Contents

List of Figures

List of Tables

1 Introduction

1.1 Background and Motivation

A query optimiser's access-path decision, whether to scan a table sequentially or to use one of several available indexes, is among the most consequential choices it makes. The problem was formalised in the System R optimiser [2] and the same cost-based framework, estimate cardinality then price each candidate path, underpins PostgreSQL today. The same query can differ by two orders of magnitude in runtime depending on that single decision.

Because the decision matters and is opaque to the user, a large body of tuning folklore has grown around it. Administrators are advised to create extended statistics when columns are correlated [3], to lower `random_page_cost` when running on solid-state storage [4], and to add BRIN indexes on naturally ordered columns because they are small and cheap to build [5]. This advice appears in official documentation, vendor engineering blogs and community forums. It is rarely accompanied by systematic measurement.

1.2 Problem Statement

Prior work established that cardinality misestimation is the dominant source of poor plans [6], [7], but studied join ordering rather than single-table access-path selection. Whether the shipped classical planner selects access paths well, and whether the remedies commonly recommended for it actually improve matters, has not been examined systematically. This report addresses that gap through four research questions.

- **RQ1.** How often does the PostgreSQL planner select a slower access path than one available to it, and what does that cost?
- **RQ2.** Which data properties drive misselection: value skew, correlation between predicate columns, or physical clustering?
- **RQ3.** Is cardinality misestimation the cause, as the optimisation literature would predict?
- **RQ4.** Do the three tuning practices named above actually improve access-path selection?

1.3 Objectives and Scope

- Build a benchmark in which skew, predicate correlation and physical clustering vary independently, with exact ground-truth cardinality for every predicate.
- Quantify PostgreSQL 17's access-path selection quality.
- Evaluate three recommended tuning practices empirically and explain any failures mechanistically.
- Publish a fully reproducible artifact.

Excluded from scope. Join ordering and join method selection, which are separate and well-studied problems [7]; cold-cache behaviour, because reliably evicting the operating system page

cache is not portable; and multi-user concurrency, since all measurements are single-session.

1.4 Report Organization

Section 2 reviews the relevant theory and prior work. Section 3 describes the experimental design and measurement protocol. Section 4 presents the results. Section 5 interprets them and states threats to validity. Section 6 concludes.

2 Background and Literature Review

2.1 Key Concepts

Access-path selection. Given a predicate over a table, the optimiser enumerates candidate access paths, a sequential scan and one path per usable index, estimates how many rows each will produce, and prices each using a cost model [2], [8]. In PostgreSQL, cost is expressed in arbitrary units anchored to `seq_page_cost`, with `random_page_cost` expressing how much more expensive a randomly fetched page is assumed to be [9].

Index types. A B-tree supports equality and range predicates and returns exact tuple pointers [10]. A hash index supports equality only. A BRIN index stores summary information per block range rather than per tuple, making it very small, but it returns *candidate block ranges* rather than rows, so matching tuples must be rechecked. Its usefulness therefore depends entirely on the physical correlation between indexed value and row position [5].

Cardinality estimation. PostgreSQL estimates the selectivity of a conjunction by multiplying the per-column selectivities, which assumes the columns are independent [11]. When they are not, the estimate can be wrong by orders of magnitude. Extended statistics objects created with `CREATE STATISTICS` are the documented remedy [3].

2.2 Related Work

Optimiser quality. Leis et al. [6], [7] showed with the Join Order Benchmark that cardinality misestimation, not cost-model calibration, dominates plan quality, and that errors compound multiplicatively through join trees. Their focus is join ordering; single-table access-path selection is not isolated. Our results are complementary and, in one respect, opposed: for the access-path decision we find misestimation is *not* the primary cause of the misselection observed.

Estimation error metrics. Moerkotte et al. [12] introduced q-error, the symmetric ratio between estimated and actual cardinality, and showed it bounds plan cost degradation. We adopt it unchanged.

Benchmarking methodology. Raasveldt et al. [13] catalogue common pitfalls in database performance testing, including failing to control caching, reporting single runs, and using mea-

surement instruments whose overhead depends on the plan being measured. Our protocol in Section 3.5 addresses each of these explicitly. Wongkham et al. [14] provide a model for rigorous index benchmarking, in particular the practice of measuring each index design in isolation rather than inferring its behaviour.

System documentation. The behaviour we report for BRIN is adjacent to a known difficulty acknowledged on the PostgreSQL hackers mailing list [15], where BRIN cost estimation is discussed as unreliable. That discussion concerns the opposite direction, BRIN being faster but not chosen. We are not aware of a systematic quantification of the direction reported here.

2.3 Research Gap

Three gaps motivate this study. First, access-path selection in the shipped classical planner has not been isolated and measured in the way join ordering has. Second, the tuning practices recommended to improve it are supported by mechanism arguments rather than workload-level measurement. Third, where effects are reported they are seldom accompanied by the conditions under which they cease to hold, so practitioners cannot tell whether a finding applies to them.

3 Methodology

3.1 Overview of Approach

The study measures, for each query, the execution time of the plan the planner freely chose against the fastest plan obtainable by restricting the available indexes. The ratio between them isolates the quality of the planner's decision from the intrinsic difficulty of the query.

Because PostgreSQL provides no mechanism to hide one index from the planner while leaving others visible, each candidate access path is measured by building that index *alone* and running the full query set. A final configuration builds every index at once, reproducing a realistic deployment, and is where the planner's actual choice is observed.

3.2 Definitions and Metrics

q-error [12] measures cardinality misestimation:

$$q(\hat{n}, n) = \frac{\max(\hat{n}, n)}{\min(\hat{n}, n)} \tag{1}$$

where $\hat{n}$ is the estimate and n the true row count, both floored at 1. A value of 1.0 is a perfect estimate. The ratio form is used because a factor-of-ten underestimate and a factor-of-ten overestimate are equally damaging to plan choice.

Access-path regret measures the cost of the decision:

$$R(q) = \frac{T_{\text{chosen}}(q)}{\min_{a \in A} T_a(q)} \tag{2}$$

where A is the set of index configurations and T is median execution time. $R = 1$ means the planner made the best available choice; $R = 4$ means the query took four times longer than necessary. Because the denominator is the fastest configuration *measured* rather than a proven optimum, reported regret is a **lower bound** on the true penalty.

Misselection is defined as $R > 1.2$. The threshold prevents run-to-run variation from being counted as a planner error; it is a judgement call and is stated as such.

3.3 Data Generation

Real datasets do not allow one property to be varied while others are held fixed, so all data is generated. Three factors are controlled independently, each varied around a uniform, independent baseline so that every effect is attributable (Table 1).

Table 1: Controlled factors. Eleven dataset configurations result, each varying one factor from the baseline.

Factor	Levels	Controls
`zipf_s`	0.0, 0.5, 1.0, 1.5	Value skew of the indexed column
`dep_strength`	0.0, 0.25, 0.5, 0.75, 1.0	Strength of the functional dependency $a \to b$
`physical_corr`	0.0, 0.5, 0.95, 1.0	Correlation of value order with row order

Domain sizes are load-bearing. With one million rows and 100 distinct values in each of columns a and b, an independent conjunction `a = va AND b = vb` returns approximately 100 rows, exactly what the independence assumption predicts. Under a full functional dependency the same predicate returns all 10,000 rows sharing `a = va` while the estimate remains at 100. That is a q-error of 100 at one percent selectivity, which is precisely the region where index and sequential access compete.

All randomness is seeded, so the corpus is byte-for-byte reproducible.

3.4 Query Families

Four families are used, each with exactly known true cardinality computed by counting the generated arrays directly rather than by asking the database:

- **Equality** on the skewed column. Candidate paths: hash, B-tree, BRIN, sequential scan.
- **Range** on the skewed column. Hash cannot serve this, which tests whether the planner falls back correctly.
- **Range on the physically clustered column.** BRIN's most favourable case.

- **Conjunctive** on the correlated pair, in dependent and independent variants. The independent variant is the control.

3.5 Measurement Protocol

Every timed execution used `EXPLAIN (ANALYZE, BUFFERS, TIMING OFF, FORMAT JSON)`, reading the top-level `Execution Time`. Three reasons, each affecting validity [13]:

1. `ANALYZE` executes server-side and discards the result set, so client transfer never enters the measurement.
2. `TIMING OFF` disables per-node instrumentation. With timing enabled, PostgreSQL reads the clock twice per emitted row, adding an overhead that is *plan-dependent*. That would systematically bias the comparison toward plans emitting fewer rows, which is the exact comparison being made.
3. Using one instrument across all configurations means residual overhead is common mode and cancels in the ratio defining regret.

Each query ran seven times; the first was discarded as cache warming and the median of the remaining six reported, with all individual runs retained. Parallelism was disabled (`max_parallel_workers_per_gather = 0`) so it could not confound the serial access-path decision, and JIT compilation was disabled because it engages on cost thresholds and would therefore fire inconsistently across configurations.

3.6 Validity Checks

Nineteen automated checks run before any measurement is trusted. Zipfian skew concentrates the top ten values from 0.3% of rows at $s = 0$ to 76.8% at $s = 1.5$; realised dependency strength lands within 0.005 of the requested value at every level; realised physical rank correlation is 0.005, 0.497, 0.949 and 1.000 for the four settings, and is *independently confirmed against PostgreSQL's own* `pg_stats` *correlation estimate* at run time; every predicate's row count matches a direct recount of the generated arrays; and identical seeds produce identical data.

One methodology defect was found and corrected during the study. The range predicate builder originally anchored a fixed-width window at the median row position. On a Zipfian column the median row falls inside a single very frequent value, so the bounds collapsed onto that value and the predicate returned its entire frequency regardless of the requested target; on the most skewed dataset every target below 13.6% produced an identical query. The affected measurements were discarded and re-collected after the fix. Recorded row counts were always exact, so no reported metric was wrong, and the headline results were essentially unchanged by the correction (73.9% to 73.3%).

Table 2: Experimental environment. Full settings are captured automatically per run in the artifact [1].

Component	Specification
CPU	Intel Core i5-12400, 6 cores / 12 threads
Memory	23.8 GB
Storage	SSD, dedicated tablespace on a device separate from the OS
Operating System	Windows 11 Pro (build 26220)
DBMS	PostgreSQL 17.1, x86_64, MSVC 19.41
`shared_buffers`	128 MB
`work_mem`	4 MB
`random_page_cost`	4.0 (default); swept 4.0 to 1.0
Table scales	1,000,000 rows (heap 112 MB, resident) and 10,000,000 rows (heap 1.1 GB, not resident)
Total executions	11,136

3.7 Experimental Environment

4 Results and Findings

4.1 RQ1: Baseline Planner Quality

With a conventional index set, that is B-tree and hash indexes but no BRIN, the planner performs well (Table 3). At one million rows it chose the best available access path for 97.7% of queries, and where it erred the median penalty was 1.3%.

Table 3: Access-path selection quality with a conventional index set (no BRIN).

Scale	Queries	Misselection	Median R	p90 R	Max R
1M	130	2.3%	1.013	1.13	1.39
10M	155	32.3%	1.043	1.52	3.70

This is a positive result and it frames everything that follows: PostgreSQL's access-path selection is not broken, so the failures reported below are specific rather than general.

4.2 RQ3: Misestimation Is Not the Cause

Of the 113 misselected queries at one million rows, the median q-error was **1.017**, and **98.2% had a q-error below 1.5**. The planner's row estimates were essentially correct and it chose badly regardless. Figure 1 plots regret against estimation error and shows no relationship of the kind the literature would predict.

This is the study's most important negative result. It rules out the explanation that the optimisation literature [6] would predict for join ordering, and redirects attention to the cost model and to path generation.

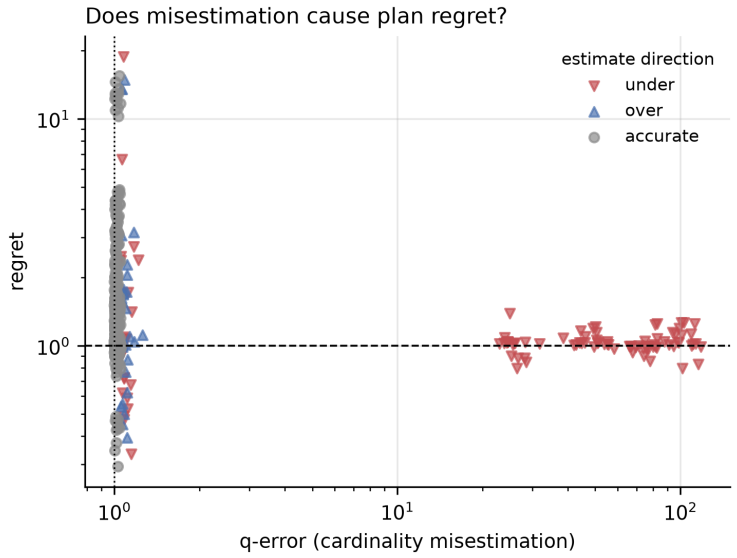

Figure 1: Access-path regret against cardinality estimation error. Points are individual queries, coloured by whether the estimate was an underestimate, an overestimate, or accurate. Regret is not explained by estimation error: the high-regret points cluster at q-error near 1.

4.3 RQ4a: Extended Statistics

`CREATE STATISTICS` is the documented remedy for correlated predicates [3]. It works, on the estimate, and harms the plan (Table 4, Figure 2).

Table 4: Effect of extended statistics on dependent conjunctive predicates, 1M rows. Estimates become nearly exact while every affected query becomes slower.

Dep.	**q-error**		**Est. rows**	**True**	**Median ms**		**Slow-**
strength	without	with	with	rows	without	with	**down**
0.25	25.23	1.11	2,767	2,556	0.93	2.54	**2.73**×
0.50	49.52	1.04	5,200	5,053	2.12	3.53	1.66×
0.75	75.89	1.02	7,633	7,513	2.80	4.34	1.55×
1.00	113.19	1.05	9,700	9,995	3.95	5.06	1.28×

Mechanism. The plans show the cause directly. Without extended statistics the planner uses the composite index on `(a, b)` in a single index scan. With them it switches to a `BitmapAnd` of two single-column indexes. Both fetch the identical 7,317 heap blocks, but the `BitmapAnd` performs two index scans plus a bitmap intersection instead of one scan. The reason for the switch is visible in the node estimates:

```
Bitmap Heap Scan  ... rows=10367     <- extended statistics applied
  -> BitmapAnd    ... rows=107       <- independence assumption, NOT
     corrected
```

Listing 1: Node row estimates with extended statistics present. The correction reaches the upper node but not the `BitmapAnd` beneath it.

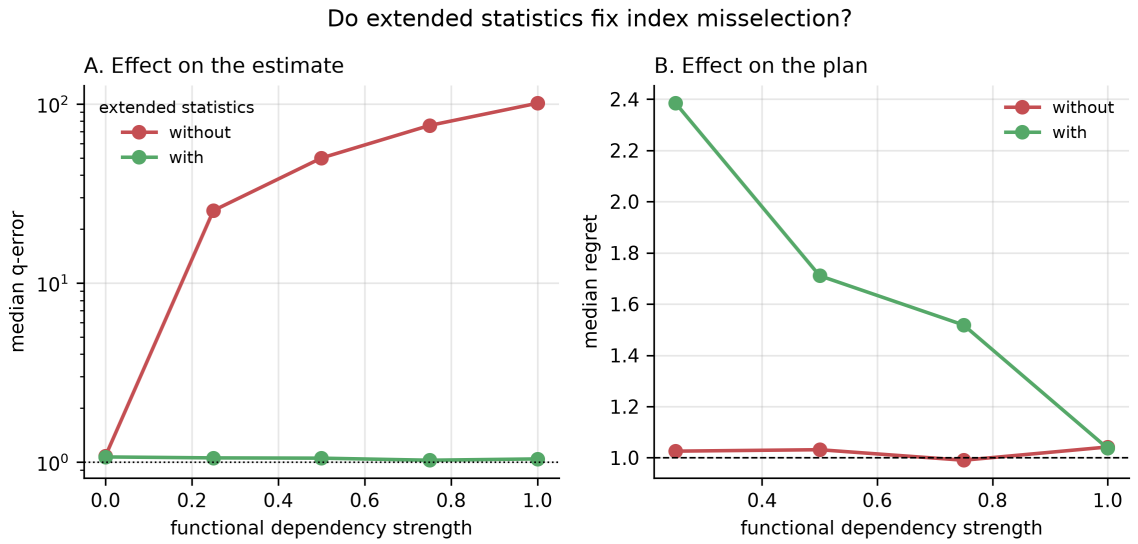

Figure 2: Extended statistics against functional dependency strength. Panel A: the estimate improves by up to two orders of magnitude. Panel B: the resulting plan gets worse. Improving the estimate and improving the plan are separable outcomes.

Since $10367 \times 10367/10^6 \approx 107$, the `BitmapAnd` node is still multiplying per-column selectivities under the independence assumption that extended statistics exist to replace. The planner therefore prices the `BitmapAnd` at 635 using the *uncorrected* figure and the composite path at 13,558 using the *corrected* one, a 21-fold gap, and selects the plan that is in fact slower. The correction is applied inconsistently across path types.

4.4 RQ4b: Lowering `random_page_cost`

The default `random_page_cost` of 4.0 assumes a random page fetch costs four times a sequential one, a spinning-disk assumption. Community guidance recommends lowering it on SSD storage. Measured across the whole query set, that advice is wrong in this environment (Table 5, Figure 3).

Table 5: Total execution time over the full query set at each `random_page_cost`, conventional index set. The default is within 6% of optimal.

`random_page_cost`	**Total time (ms)**	**Versus best**
1.0	2,306	**2.3× worse**
1.1	1,954	1.9× worse
1.2	1,942	1.9× worse
1.5	**1,006**	best
2.0	1,053	1.0×
3.0	1,053	1.0×
4.0 (default)	**1,068**	1.06×

The access-path counts explain why. At `random_page_cost = 1.0` the planner aban-

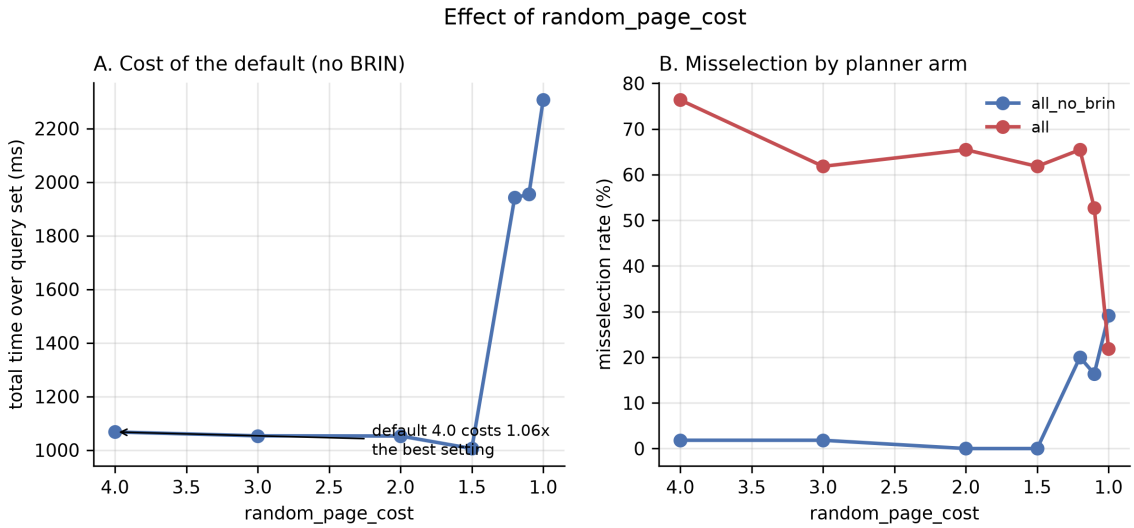

Figure 3: Effect of `random_page_cost`. Panel A: total query-set time; lowering the setting below 1.5 degrades the workload. Panel B: misselection rate by index configuration.

dons bitmap scans entirely in favour of plain index scans, 136 index scans and zero bitmap scans, which is the wrong choice for higher-selectivity predicates.

A single-query measurement early in this study suggested the default cost 24 times (Appendix A, Table 11). That observation was real but unrepresentative, and the workload-level sweep supersedes it. It is retained deliberately as a caution against tuning from individual queries.

4.5 RQ2 and RQ4c: A Co-located BRIN Index

BRIN indexes are small and cheap to build, so adding one is often treated as harmless. It is not, while the table is buffer-pool resident (Table 6, Figure 4).

Table 6: Effect of adding a BRIN index alongside an existing B-tree on the same column.

Scale	With BRIN	Without BRIN	Median R	Max R
1M	**73.3%**	2.3%	2.06 vs 1.01	**18.78**
10M	30.1%	32.3%	1.04 vs 1.04	2.63

At one million rows, adding a BRIN index alongside an existing B-tree raises misselection from 2.3% to 73.3%. In controlled diagnostics with statistics held byte-identical, individual queries degraded by up to **194 times**, from 0.34 ms to 65.95 ms.

Mechanism. With both indexes present and a bitmap plan forced, the planner selected BRIN in **16 of 16 cases**, and in ten of those the B-tree bitmap path was cheaper *by the planner's own cost model*, once by a factor of nine. A cost-based optimiser does not knowingly select a path it prices as nine times more expensive, which indicates the B-tree bitmap path is discarded

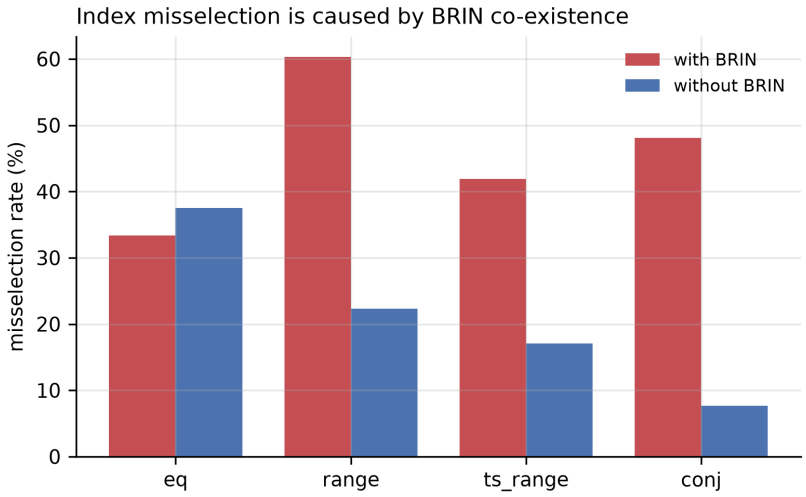

Figure 4: Misselection rate by query family with and without a BRIN index in the index set, 1M rows. The conjunctive family is the control: no BRIN index touches those columns, and correspondingly there is no effect.

during path generation rather than rejected on cost. This is an inference from cost arithmetic and has not been confirmed against PostgreSQL's source.

The cost of the chosen plan is visible in its execution: `Heap Blocks: lossy=14304` with `Rows Removed by Index Recheck: 990000`. BRIN reads essentially the whole table and discards 99% of what it reads.

Figure 5 shows how each index type responds to the physical correlation BRIN depends on.

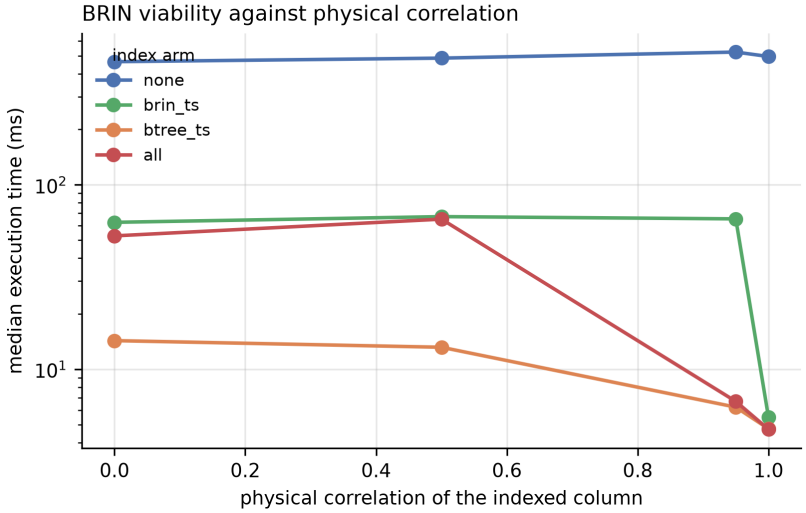

Figure 5: Execution time against physical correlation of the indexed column for each index configuration. BRIN becomes viable only as correlation approaches 1.

4.6 Locating the Boundary

Comparing one million against ten million rows shows the penalty present at the smaller scale and absent at the larger, but two points cannot distinguish a gradual decay from a sharp transition. We therefore repeated the measurement at five intermediate scales, using the `phys` datasets so the comparison is like for like (Table 7, Figure 6).

Table 7: The BRIN penalty across table size. Two quantities move independently: how *often* the planner errs, and how *badly* it errs in the worst case.

Million rows	Heap (MB)	Blocks read outside pool	Misselection %		Max regret	
			with BRIN	without	with BRIN	without
1.00	111.8	0	**71.4**	0.0	18.78	1.06
1.25	139.8	0	37.9	34.5	**38.81**	1.67
1.50	167.8	0	31.0	17.2	**40.32**	1.76
2.00	223.2	0	25.8	12.9	**38.34**	1.77
3.00	335.2	0	30.3	12.1	**41.51**	1.67
5.00	558.2	1,902	36.4	30.3	6.98	1.81
10.00	1,116.2	89,081	38.2	44.4	1.92	3.70

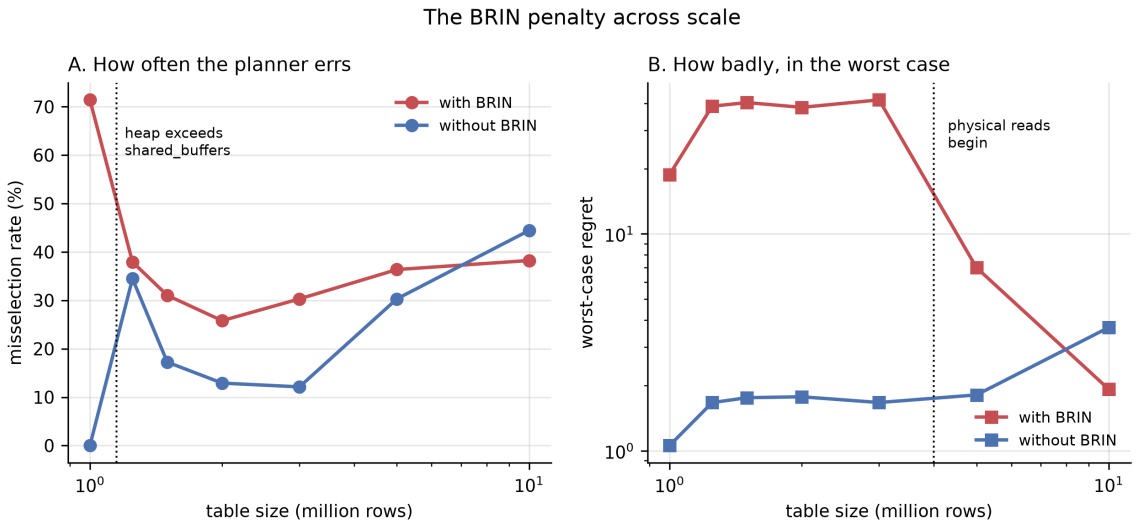

Figure 6: The BRIN penalty across scale. Panel A: how often the planner errs. Panel B: worst-case regret. The two collapse at different table sizes, near two different system boundaries.

The transition is sharp, and there are *two* of them.

Frequency collapses at the buffer pool boundary. The gap in misselection rate falls from 71.4 percentage points at one million rows to 3.4 at 1.25 million. That is precisely where the heap crosses `shared_buffers`: 111.8 MB against the 128 MB pool at one million rows, 139.8 MB at 1.25 million. Note what drives the collapse: the with-BRIN configuration improves from 71.4% to 37.9%, but the conventional configuration *degrades* from 0.0% to 34.5%. The gap closes largely because the baseline gets worse, not because BRIN gets better.

Severity collapses much later, and first gets worse. Worst-case regret with BRIN does not decline past the buffer pool boundary. It roughly doubles, from 18.78 at one million rows to 38.81 at 1.25 million, and stays near 40 through three million. It falls only at five million (6.98) and ten million (1.92). That later collapse coincides with the onset of physical reads: blocks fetched from outside the buffer pool are zero up to three million rows, 1,902 at five million and 89,081 at ten million.

This corrects a conclusion that two-point comparison would have supported. The BRIN penalty does not simply vanish with scale. Its *frequency* falls once the table stops fitting in the buffer pool, while its *worst case* intensifies over the next several-fold increase in table size and subsides only once queries begin paying real I/O. A practitioner whose table is two or three times the buffer pool faces the most severe individual regressions measured anywhere in this study.

We do not claim a verified causal mechanism for either transition. What is established is that each coincides with a specific, measurable system boundary, and that the two boundaries are different.

4.7 Incidental Finding: Hash Index Build Cost Under Skew

Hash index build time proved highly sensitive to value skew (Table 8), a 185-fold blowup from uniform to heavily skewed data while a B-tree on the same column is unaffected. Zipfian values collide into few hash buckets, producing long overflow chains, and PostgreSQL does not parallelise hash builds as it does B-tree builds.

Table 8: Index build time at 10M rows by value skew.

Dataset	`zipf_s`	**Hash build**	**B-tree build**
baseline	0.0	11.6 s	3.2 s
skew05	0.5	16.7 s	2.3 s
skew10	1.0	187.7 s	2.5 s
skew15	**1.5**	**2,152.9 s**	2.3 s

4.8 Analysis and Interpretation

Figure 7 places the results in context by plotting regret against true selectivity. Misselection concentrates in the one to ten percent selectivity band, which is exactly where the cost of an index scan and a sequential scan converge and where small cost-model errors therefore flip the decision.

The mechanism is consistent across all three failures. A bitmap heap scan's cost is dominated by the number of heap pages it expects to fetch, priced at `random_page_cost`. Any error in that estimate, whether from a lossy BRIN bitmap, an uncorrected `BitmapAnd` node,

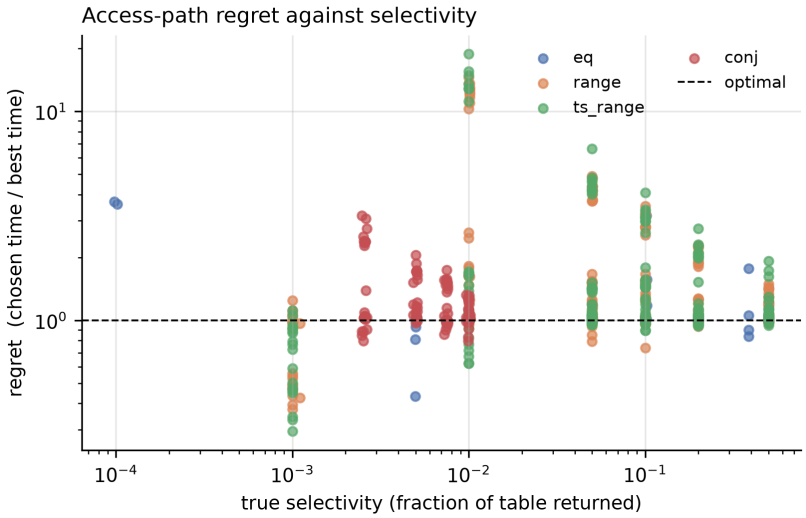

Figure 7: Access-path regret against true selectivity, by query family. The dashed line at $R = 1$ marks optimal choice.

or a miscalibrated page-cost constant, moves the crossover point between index and sequential access. Because the true execution times on either side of that crossover differ sharply, a small pricing error produces a large runtime penalty.

5 Discussion

5.1 Key Insights

The three practices fail for the same underlying reason: each is justified by reasoning about a single quantity, while plan quality depends on several.

Extended statistics optimise **estimate accuracy**, and a more accurate estimate improves the plan only if applied consistently wherever the plan is costed. It is not.

Lowering `random_page_cost` optimises for **storage latency**, and it is true that random access on SSD is cheaper than the default assumes. But the same constant governs the choice between bitmap and plain index scans, so pushing it too low wins on one axis and loses on another.

Adding a BRIN index optimises for **index size and build cost**, both of which genuinely favour BRIN. But index availability also changes which paths the planner generates, and a cheap index that captures the bitmap path can exclude a better one.

In each case the advice is correct about the quantity it names and wrong about the outcome, because the outcome depends on more than that quantity. This is a general caution about mechanism-based tuning advice that has not been validated at the workload level.

5.2 Connection to Course Concepts

The work applies several core topics from the course directly. *Indexing*: the trade-off between B-tree, hash and BRIN structures is measured rather than asserted, including the dependence of BRIN on physical clustering and of hash on value distribution. *Query processing and optimisation*: the System R cost-based framework [2] is exercised at its weakest point, the transition between access paths, and the propagation of selectivity estimates through a plan tree is observed directly in Listing 1. *Storage management*: the buffer pool residency boundary in Section 4 is precisely the distinction between logical reads served from `shared_buffers` and physical reads issued beyond it, and it turns out to determine whether an entire finding holds.

5.3 Recommendations for Practitioners

1. Do not add extended statistics reflexively. Verify that the *plan* improved, not merely the estimate. The two are separable.
2. Do not lower `random_page_cost` below approximately 1.5 without measuring. The default was within 6% of optimal here.
3. Treat a BRIN index alongside a B-tree on the same column as a risk to be measured, particularly where the table fits in the buffer pool.
4. Do not assume hash index creation is cheap on skewed columns.

5.4 Limitations and Threats to Validity

Internal validity. Regret is a lower bound, since the denominator is the fastest measured configuration rather than a proven optimum. Values slightly below 1.0 occur because the planner can combine indexes in ways no single restricted configuration can. The 1.2 misselection threshold is a judgement call.

Construct validity. The path-suppression mechanism in Section 4 is inferred from cost arithmetic, not confirmed against PostgreSQL's source code. The extended statistics result requires a redundant index set, a composite index plus both single-column indexes, which is common in practice but not universal.

External validity. One DBMS, one version, one hardware configuration. All measurements are warm-cache; reliably evicting the operating system page cache is not portable across platforms, so cold-storage behaviour is out of scope rather than approximated. With 23.8 GB of RAM the page cache retains the data even at ten million rows, so the larger scale tests leaving the *buffer pool*, not leaving memory. Single-table access paths only.

6 Conclusion and Recommendations

PostgreSQL 17's access-path selection is close to optimal under a conventional index set, choosing the best available plan for 97.7% of the queries measured. Where it fails, cardinality misestimation is not the cause: 98.2% of misselected queries had accurate row estimates. This contrasts with the established result for join ordering [6] and suggests the access-path decision has a different failure profile.

The failures instead trace to the cost model and to path generation, and are triggered by three practices intended to help. Extended statistics improve estimates by up to 108 times and slow queries by up to 2.7 times, because the correction is not propagated to the `BitmapAnd` node. Lowering `random_page_cost` on SSD, contrary to common advice, costs a factor of 2.3 in this workload, the default being within 6% of optimal. A co-located BRIN index raises misselection from 2.3% to 73.3% and can slow individual queries by 194 times while the table is buffer-pool resident.

The boundaries are as much a part of that last result as the effect itself, and measuring them changed what we could claim. A two-point comparison, one million rows against ten million, would have supported the conclusion that the penalty simply vanishes with scale. Five intermediate scales show otherwise: its frequency collapses once the heap exceeds `shared_buffers`, but its worst case roughly doubles at that same point and subsides only once queries begin paying physical I/O, several-fold further out. The most severe individual regressions in this entire study occur at two to three times the buffer pool size, not at the smallest scale.

That is the general lesson. A tuning recommendation is not right or wrong in isolation; it is right or wrong under conditions, and those conditions are measurable. Reporting an effect without its boundary invites practitioners to apply it where it does not hold, or to dismiss it where it does.

Future work. Three directions would strengthen these findings: confirming the path-suppression mechanism against PostgreSQL's source code; repeating the study on a second DBMS to test whether the results are PostgreSQL-specific; and extending to cold-cache conditions on hardware where page cache eviction is controllable, which would separate buffer pool residency from operating system page cache residency, a distinction the present hardware cannot make.

7 References

[1] I. Sajin. "Access-path selection benchmark for PostgreSQL: Code, raw measurements and analysis," Accessed: Aug. 16, 2026. [Online]. Available: https://github.com/Imtiaj-Sajin/Reseach-on-Database-Architecture

[2] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, "Access path selection in a relational database management system," in *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*, 1979, pp. 23–34. DOI: 10.1145/582095.582099

[3] PostgreSQL Global Development Group. "Postgresql 17 documentation: Create statistics," Accessed: Aug. 16, 2026. [Online]. Available: https://www.postgresql.org/docs/17/sql-createstatistics.html

[4] PostgreSQL Global Development Group. "Postgresql 17 documentation, section 19.7: Query planning configuration," Accessed: Aug. 16, 2026. [Online]. Available: https://www.postgresql.org/docs/17/runtime-config-query.html

[5] PostgreSQL Global Development Group. "Postgresql 17 documentation, chapter 68: Brin indexes," Accessed: Aug. 16, 2026. [Online]. Available: https://www.postgresql.org/docs/17/brin.html

[6] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, "How good are query optimizers, really?" *Proceedings of the VLDB Endowment*, vol. 9, no. 3, pp. 204–215, 2015. DOI: 10.14778/2850583.2850594

[7] V. Leis et al., "Query optimization through the looking glass, and what we found running the join order benchmark," *The VLDB Journal*, vol. 27, no. 5, pp. 643–668, 2018. DOI: 10.1007/s00778-017-0480-7

[8] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill, 2019.

[9] PostgreSQL Global Development Group. "Postgresql 17 documentation, section 63.6: Index cost estimation functions," Accessed: Aug. 16, 2026. [Online]. Available: https://www.postgresql.org/docs/17/index-cost-estimation.html

[10] G. Graefe, "Modern b-tree techniques," *Foundations and Trends in Databases*, vol. 3, no. 4, pp. 203–402, 2011. DOI: 10.1561/1900000028

[11] PostgreSQL Global Development Group. "Postgresql 17 documentation, section 14.2: Statistics used by the planner," Accessed: Aug. 16, 2026. [Online]. Available: https://www.postgresql.org/docs/17/planner-stats.html

[12] G. Moerkotte, T. Neumann, and G. Steidl, "Preventing bad plans by bounding the impact of cardinality estimation errors," *Proceedings of the VLDB Endowment*, vol. 2, no. 1, pp. 982–993, 2009. DOI: 10.14778/1687627.1687738

[13] M. Raasveldt, P. Holanda, T. Gubner, and H. Mühleisen, "Fair benchmarking considered difficult: Common pitfalls in database performance testing," *Proceedings of the 7th International Workshop on Testing Database Systems (DBTest)*, pp. 1–6, 2018. DOI: 10.1145/3209950.3209955

[14] C. Wongkham, B. Lu, C. Liu, Z. Zhong, E. Lo, and T. Wang, "Are updatable learned indexes ready?" *Proceedings of the VLDB Endowment*, vol. 15, no. 11, pp. 3004–3017, 2022. DOI: 10.14778/3551793.3551848

[15] PostgreSQL Hackers Mailing List. "Brin index which is much faster never chosen by planner," Accessed: Aug. 16, 2026. [Online]. Available: https://www.postgresql.org/message-id/20191015224047.GV3599%40telsasoft.com

A Query Execution Plans

All plans below are unmodified `EXPLAIN` output captured during the study. The script producing each is named, so any can be regenerated from the artifact [1].

A.1 The BRIN Effect With Statistics Held Constant

Source: `diagnose_brin.py`. 1M rows, `physical_corr = 0.5`. Both indexes exist on `ts`. `ANALYZE` was run once and never again, so both plans below come from byte-identical statistics. The generator's realised rank correlation was 0.4984 and PostgreSQL's own `pg_stats` estimate 0.49863, confirming the intended clustering.

```
Bitmap Heap Scan on t_brindiag  (cost=14.56..14858.18 rows=9705) (actual
   rows=10000)
  Recheck Cond: ((ts >= 1655753994) AND (ts <= 1657336369))
  Rows Removed by Index Recheck: 990000
  Heap Blocks: lossy=14304
  Buffers: shared hit=2696 read=11616
  ->  Bitmap Index Scan on ix_brindiag_brin_ts  (cost=0.00..12.13 rows
     =35975)
Execution Time: 132.691 ms
```

Listing 2: Plan chosen with both indexes available.

```
Bitmap Heap Scan on t_brindiag  (cost=215.87..14045.79 rows=10092) (actual
   rows=10000)
  Recheck Cond: ((ts >= 1655753994) AND (ts <= 1657336369))
  Heap Blocks: exact=4293
  Buffers: shared hit=4293 read=30
  ->  Bitmap Index Scan on ix_brindiag_btree_ts  (cost=0.00..213.35 rows
     =10092)
Execution Time: 3.416 ms
```

Listing 3: Same query, same statistics, BRIN index removed.

Note `lossy=14304` against `exact=4293`, and 132.691 ms against 3.416 ms. The planner priced the chosen plan at 14,858 and the unchosen one at 14,046, so the plan it did not use was the cheaper one by its own model.

A.2 Path Suppression Across the Correlation Range

Source: `diagnose_suppression.py`. Bitmap plans forced by disabling all other access methods, statistics held constant within each row.

The largest observed penalty is the 0.95 correlation, 0.1% range case: 62.07 ms against 0.09 ms, a factor of 690.

Table 9: BRIN was selected in 16 of 16 cases. In ten the B-tree bitmap path was cheaper by the planner's own cost model.

Corr.	Query	BRIN cost	BRIN ms	B-tree cost	B-tree ms
0.0	0.1% range	29,320	65.95	3,166	**0.34**
0.0	1% range	29,322	66.73	13,857	5.14
0.0	5% range	29,332	69.50	16,111	12.87
0.5	0.1% range	15,338	66.33	3,174	**0.21**
0.5	1% range	14,852	63.90	13,848	2.76
0.95	0.1% range	13,511	62.07	3,294	**0.09**
0.95	1% range	15,365	66.45	14,201	0.76
1.0	0.1% range	13,286	0.55	2,903	0.07

A.3 Extended Statistics Change the Plan for the Worse

Source: `diagnose_extstats.py`. 1M rows, `dep_strength = 1.0`, query `SELECT * FROM t_ext WHERE a = 57 AND b = 84`, true rows 10,216. Identical table, data, indexes and session; only the extended statistics object is added.

```
Bitmap Heap Scan on t_ext  (cost=5.38..356.28 rows=93) (actual rows=10216)
  Heap Blocks: exact=7317
  ->  Bitmap Index Scan on ix_ext_ab  (cost=0.00..5.35 rows=93)
        Index Cond: ((a = 57) AND (b = 84))
```

Listing 4: Without extended statistics. Median of 15 runs: 4.059 ms (stdev 0.473).

```
Bitmap Heap Scan on t_ext  (cost=219.84..559.77 rows=9467) (actual rows
   =10216)
  Heap Blocks: exact=7317
  ->  BitmapAnd  (cost=219.84..219.84 rows=90)
        ->  Bitmap Index Scan on ix_ext_b  (cost=0.00..107.43 rows=9467)
        ->  Bitmap Index Scan on ix_ext_a  (cost=0.00..107.43 rows=9467)
```

Listing 5: With extended statistics. Median of 15 runs: 5.171 ms (stdev 0.261).

The two timing distributions do not overlap, so this is not measurement noise. Both plans fetch the identical 7,317 heap blocks.

A.4 Confirming the Planner Acts Rationally on Its Own Model

Source: `diagnose_composite.py`. With correct estimates the planner prices the composite path at 13,558 and the `BitmapAnd` at 635, a 21-fold gap, while the composite path is in fact the faster of the two (Table 10). Without extended statistics the same composite path is priced at 392.80 and runs in 4.155 ms: the *correct* estimate makes the plan appear 34 times more expensive while running at the same speed.

Table 10: Plan cost and measured time by available index set, extended statistics present.

Indexes available	**Plan chosen**	**Total cost**	**Median ms**
all three	`BitmapAnd` of a, b	635.63	4.794
composite only	composite (a, b)	13,558.73	**4.030**
separate only	`BitmapAnd`	643.90	5.357

A.5 Cost Model Sensitivity on a Single Query

Source: `diagnose.py`. Same query, same data, identical row estimate (10,662) throughout; only `random_page_cost` changes.

Table 11: Single-query sensitivity to `random_page_cost`. Taken alone this suggests the default is catastrophic; measured across the whole query set (Table 5) it is within 6% of optimal.

`random_page_cost`	**Plan chosen**	**Median ms**
4.0 (default)	sequential scan	49.14
2.0	B-tree index scan	**2.08**
1.5	B-tree index scan	2.33
1.1	B-tree index scan	2.49
1.0	B-tree index scan	2.25

A.6 Buffer Pool Residency

Median buffer counters for range queries on the physically clustered column, full index set. `shared_read` counts blocks fetched from outside the buffer pool: zero at one million rows confirms full residency, 89,081 at ten million confirms the working set has left the pool. This is the independent evidence that the two scales are genuinely different regimes.

Table 12: Buffer statistics by scale.

Scale	`shared_hit`	`shared_read`
1,000,000 rows	13,194	**0**
10,000,000 rows	11,314	**89,081**

B Reproducibility

All code, raw measurements, figures and an executable analysis notebook are published [1]. No dataset is shipped: all data is generated from a fixed seed, so the corpus is byte-for-byte identical on any machine.

```
pip install -r research/requirements.txt
psql -c "CREATE TABLESPACE ts_dtms LOCATION '/path/with/space';"
```

```
python research/tests/test_generator.py         # validate the generator
python research/tests/test_range_targeting.py   # validate query
   construction
python research/run_experiment.py --rows 1000000
python research/run_experiment.py --rows 10000000
python research/run_rpc_sweep.py
python research/analyze.py                      # regenerates every figure
```

Listing 6: Complete reproduction sequence.

Runs are resumable at the granularity of a single measurement: every result is appended to disk as it is taken, and the resume key is (dataset, index configuration, query, extended-statistics state). This was exercised twice during the study, once by a stopped process and once by a power failure; both times the loss was a single in-flight query.

Each measurement record retains all individual runs, not only the median, so the variance claims in this report can be checked independently.

C Individual Contribution Statement

Table 13: Individual contributions.

Member	**Contribution**
Imtiaj Sajin	Experimental design; benchmark implementation; data generation and validation; measurement infrastructure; diagnostics; analysis; all sections of this report.